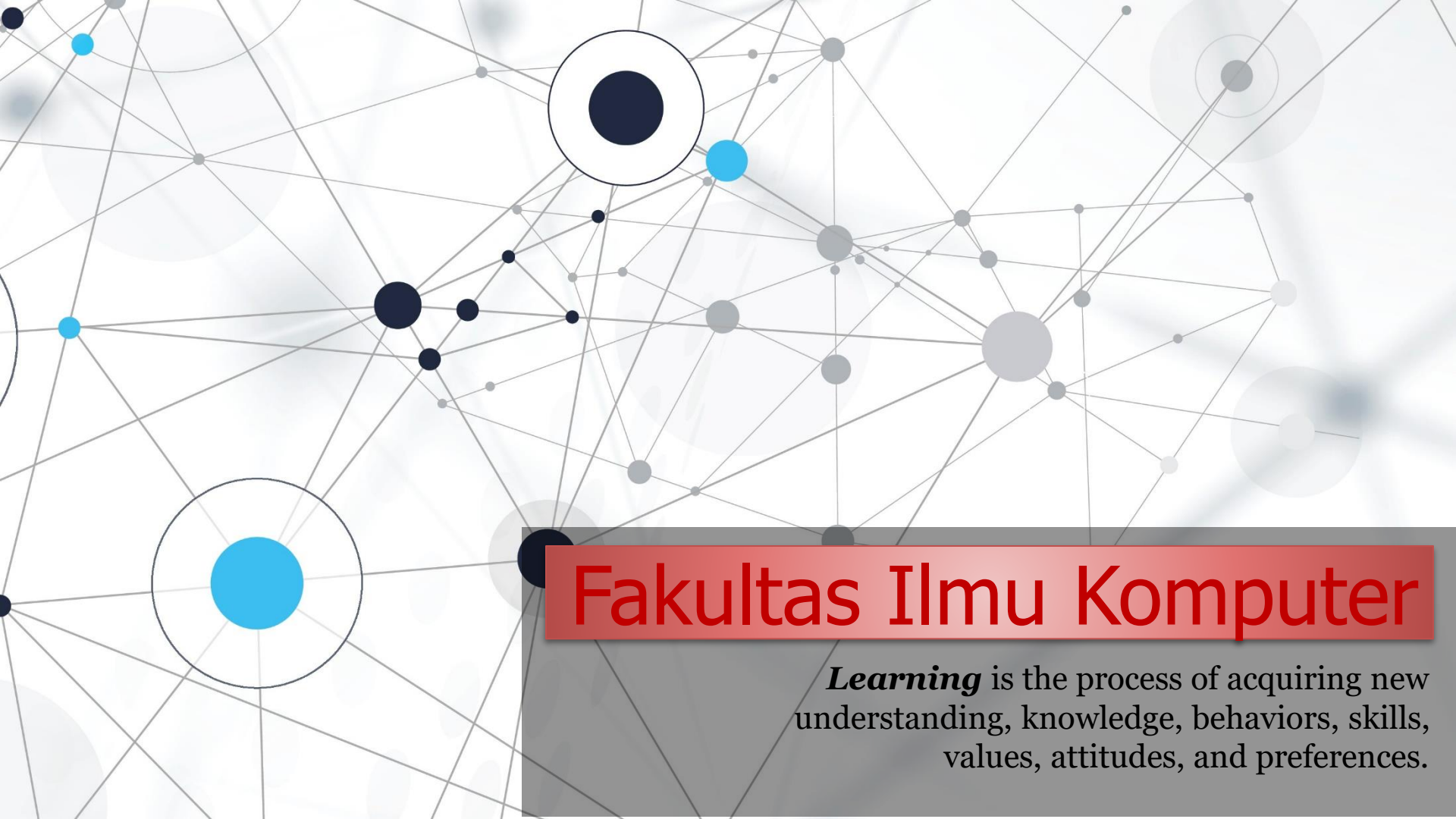


An abstract network diagram with various sized nodes (black, blue, and grey) connected by thin grey lines. Some nodes are highlighted with larger circles. The background is white with faint grey circles.

Fakultas Ilmu Komputer

Learning is the process of acquiring new understanding, knowledge, behaviors, skills, values, attitudes, and preferences.

Composition Concept *in* Python OOP



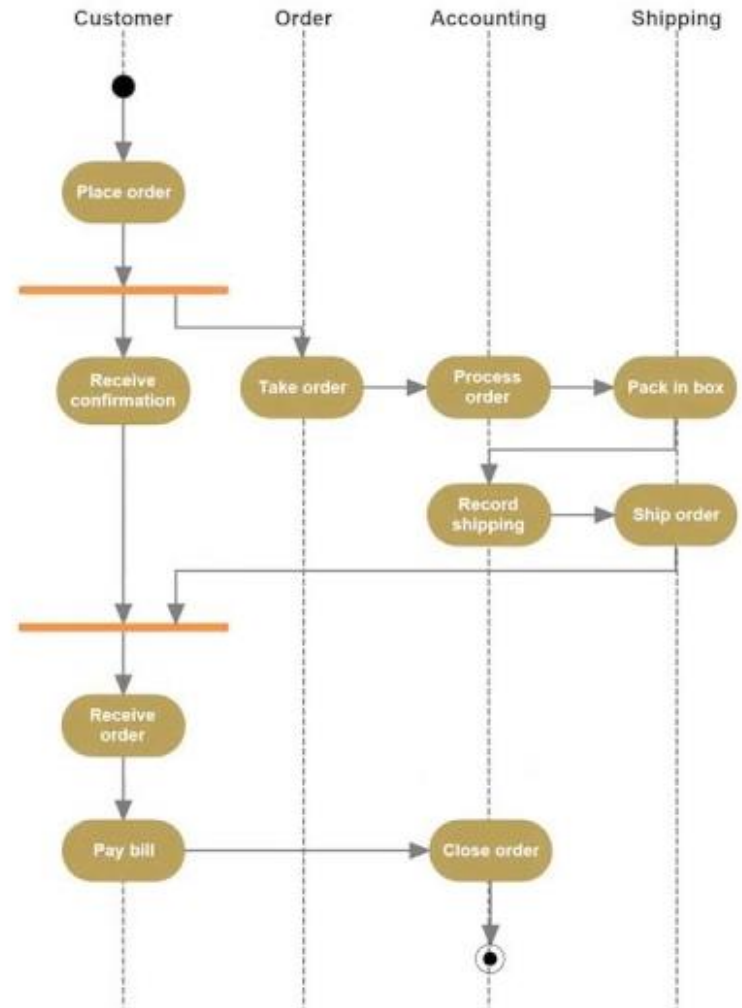
**Welcome to *the* Object Oriented
Programming *course***

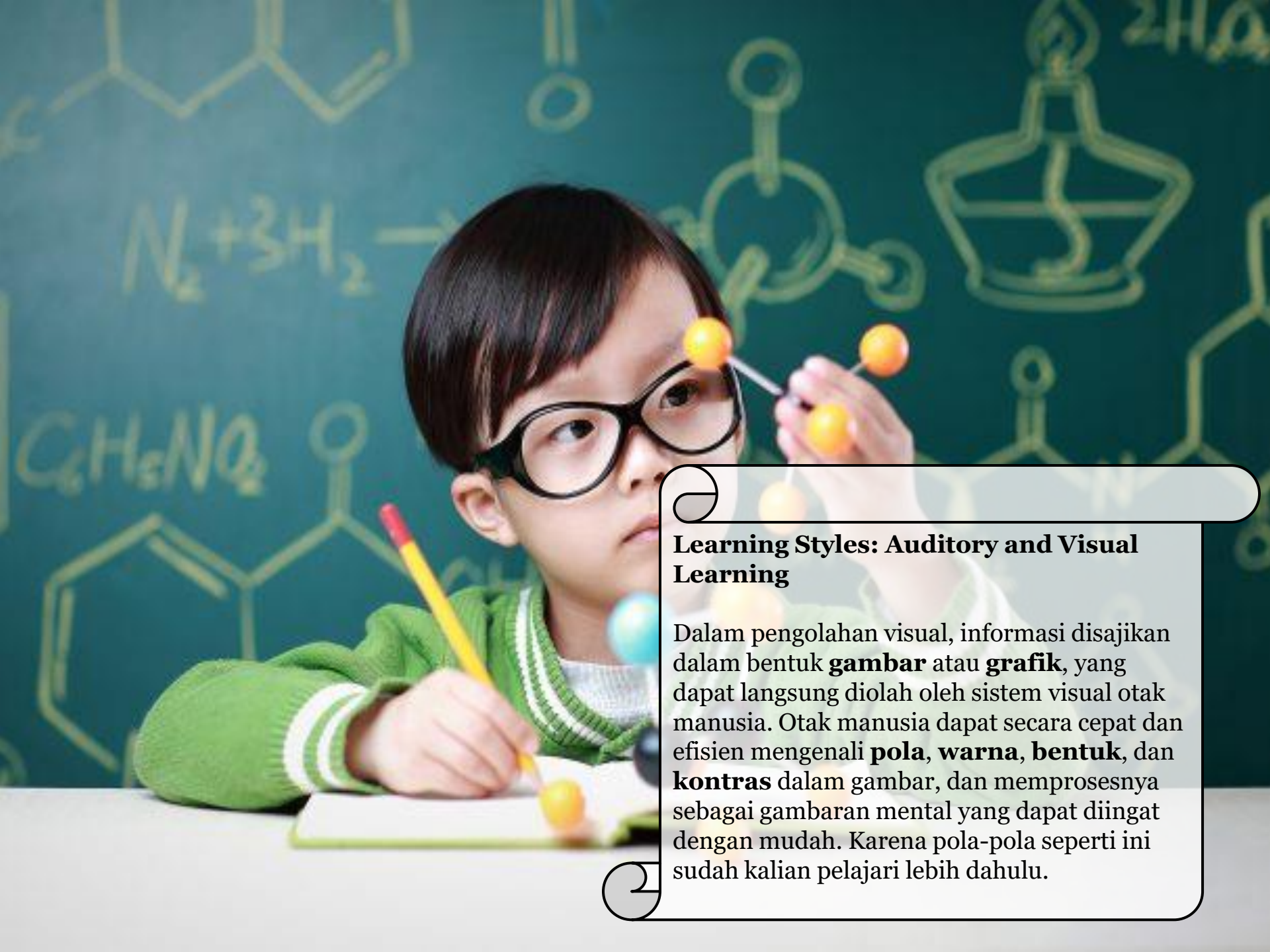
A decorative border of green leaves and branches at the top of the slide.

1 – What *is* UML *in* OOP ?

A decorative border of green leaves and branches at the bottom of the slide.

A cartoon illustration of a man with brown hair, wearing a black suit, white shirt, and blue tie. He is smiling and waving with his right hand.





Learning Styles: Auditory and Visual Learning

Dalam pengolahan visual, informasi disajikan dalam bentuk **gambar** atau **grafik**, yang dapat langsung diolah oleh sistem visual otak manusia. Otak manusia dapat secara cepat dan efisien mengenali **pola**, **warna**, **bentuk**, dan **kontras** dalam gambar, dan memprosesnya sebagai gambaran mental yang dapat diingat dengan mudah. Karena pola-pola seperti ini sudah kalian pelajari lebih dahulu.



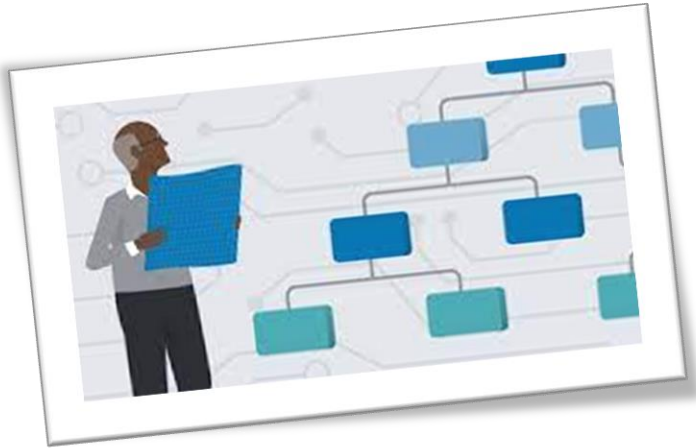
Dalam pengolahan textual (*verbal*), informasi disajikan dalam bentuk kata-kata atau teks, yang memerlukan **pemrosesan lebih lanjut oleh human brain** untuk memahami maknanya. Human brain harus menerjemahkan kata-kata menjadi gambaran mental yang dapat dipahami dan diingat.

Therefore, when learning, pictures/images/diagrams can help the human brain to process and remember information more effectively than text.



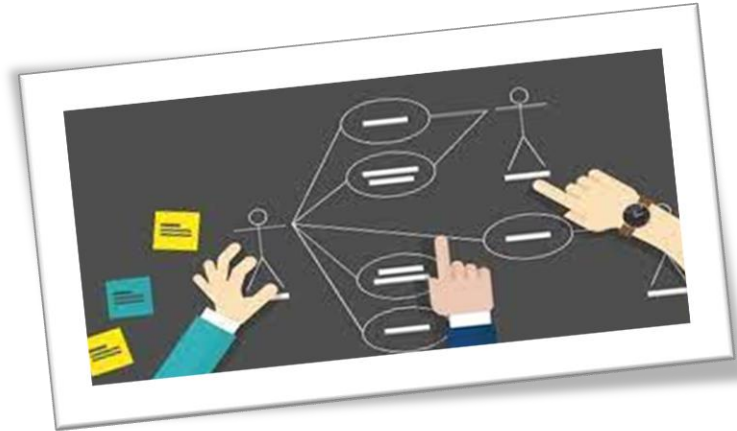
In software development , programmers also need visual processing where information is presented in the form of images or graphics to help process and remember information more effectively, using UML.

What is UML in OOP???



The Unified Modeling Language (UML) is a general-purpose modeling language that is intended to provide a standard way **to visualize the design** of a system.

UML (Unified Modelling Language) adalah suatu **metode** dalam **pemodelan secara visual** yang digunakan sebagai sarana perancangan sistem berorientasi objek.



UML menyediakan notasi dan konvensi standar untuk menggambarkan **struktur** (*structure*), **perilaku** (*behavior*), dan **interaksi** (*interaction*) antara komponen dalam sistem software (*perangkat lunak*).

UML mencakup sejumlah diagram, termasuk diagram **use case**, **diagram kelas**, **diagram aktivitas**, sequence diagram, **diagram komponen**, dll.

What is UML in OOP???

Dalam UML, sebuah sistem perangkat lunak direpresentasikan sebagai **kumpulan objek-objek dan hubungan antar objek-objek** tersebut. Dengan menggunakan UML, pengembang perangkat lunak dapat merancang dan memodelkan sistem perangkat lunak secara terstruktur dan terdokumentasi dengan jelas.

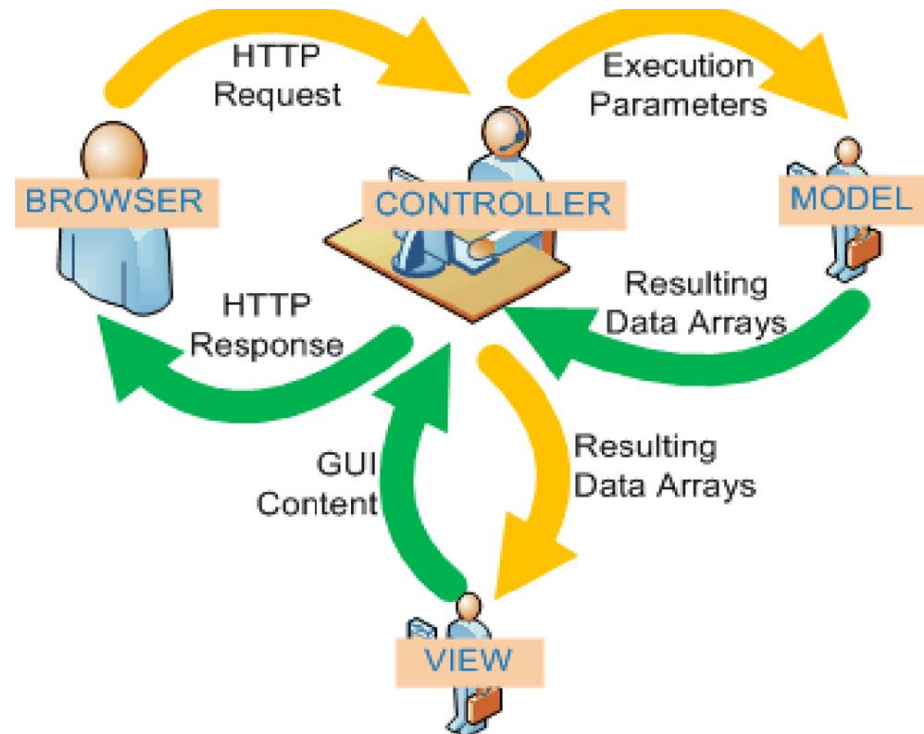


Using UML can also help us **understand the complexity** of the system and facilitate the process of developing, maintaining and evolving software systems.

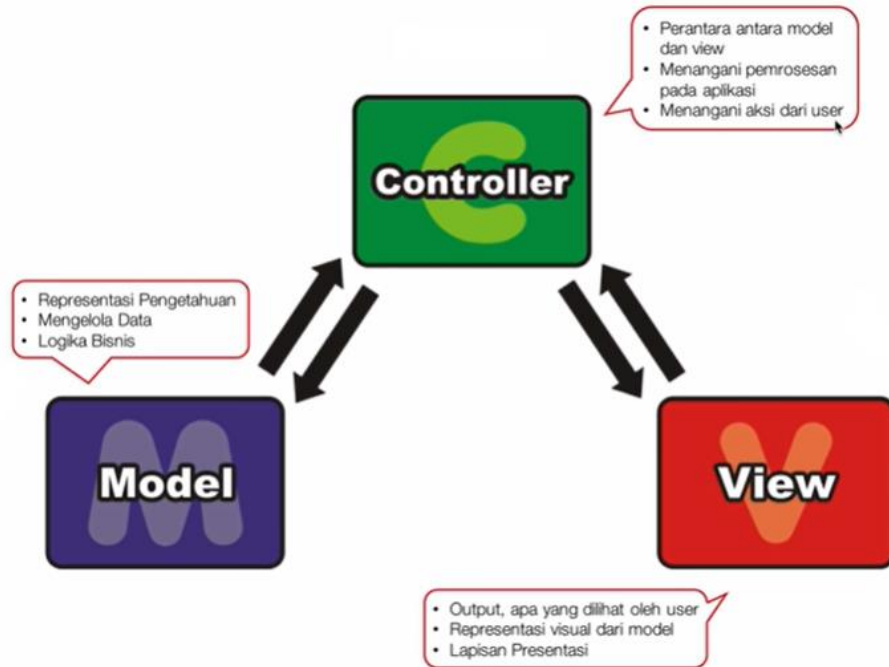
Why MVC Framework ???

Association, aggregation dan composition adalah tiga konsep penting dalam pemrograman berorientasi objek (OOP) yang digunakan untuk **menggambarkan hubungan antara objek** di dalam sistem. **Hubungan antara satu class dengan class yang lain.**

MVC Framework

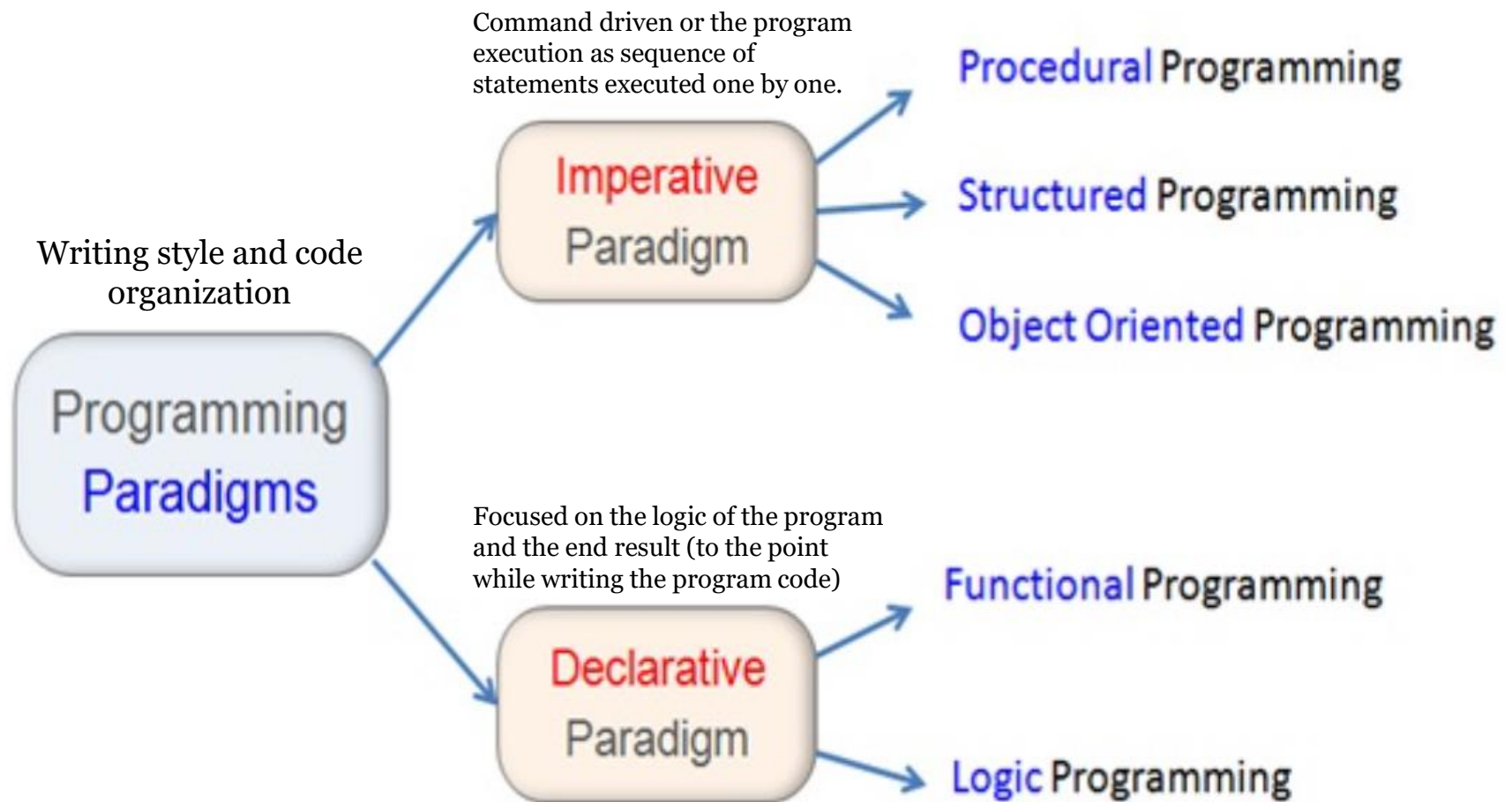


MVC Framework



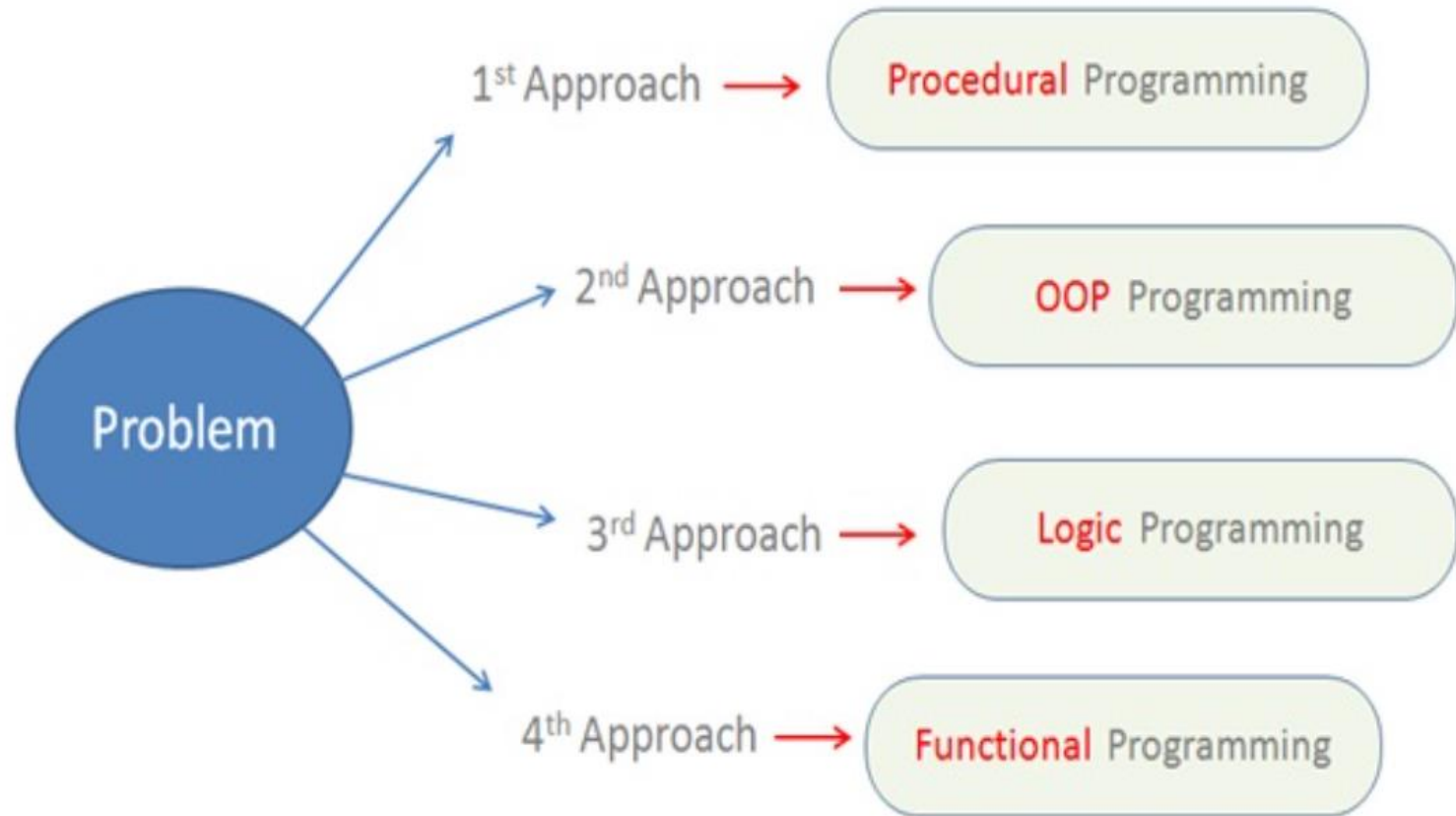
MVC (Model-View-Controller) adalah sebuah konsep yang digunakan dalam pengembangan perangkat lunak yang terpisah antara tiga komponen utama yaitu Model, View, dan Controller. Konsep ini dapat diimplementasikan dalam pendekatan berorientasi objek (OOP) untuk mempermudah pengembangan perangkat lunak yang kompleks dan memudahkan proses maintain.

- ✓ Dalam MVC, **Model adalah bagian dari aplikasi yang mengatur data dan bisnis logic**, **View adalah bagian dari aplikasi yang mengatur presentasi dan tampilan antarmuka pengguna**, sedangkan **Controller adalah bagian dari aplikasi yang mengatur interaksi antara View dan Model**.
- ✓ Contoh framework berbasis MVC dalam pendekatan OOP adalah Laravel, yang merupakan framework PHP open-source yang populer dan banyak digunakan oleh developer web. Laravel memungkinkan pengembangan aplikasi web yang mudah, cepat, dan efisien dengan menggunakan prinsip-prinsip OOP dan arsitektur MVC.



Apakah di semua paradigma bahasa pemrograman (writing style and code organization, pattern, model programming or set of rules) bisa diterapkan standar notasi UML?

Writing Style & Code Organization



The programming paradigm is all about different approaches used to find a software solution to the problem. Each solution to the problem has its own advantages and disadvantages.

Procedural Programming *with* UML?

- ❖ Jika kode yang ditulis dalam paradigma prosedural, maka **tidak ada kelas (class)** dalam arti sebenarnya. Namun, kita masih dapat menggambarkan komponen-komponen utama dalam kode menggunakan UML activity diagram, sequence diagram, atau component diagram.
- ❖ Kita dapat menggambarkan fungsi-fungsi (*functions*) atau prosedur (*procedures*) dalam **activity diagram** atau **sequence diagram** dengan menunjukkan **aliran data** antara fungsi atau prosedur.
- ❖ Kita juga dapat menunjukkan bagaimana fungsi atau prosedur saling berinteraksi dalam **sequence diagram**.
- ❖ Kita juga dapat menggunakan **UML component diagram** untuk menunjukkan bagaimana komponen-komponen dalam sistem berinteraksi satu sama lain. Komponen-komponen dapat berupa *file*, *modul*, atau bagian-bagian dari *kode*.

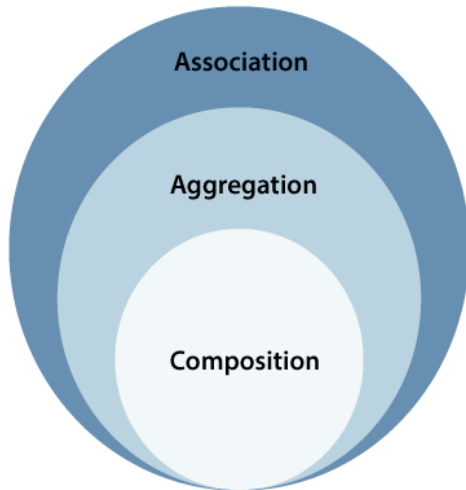
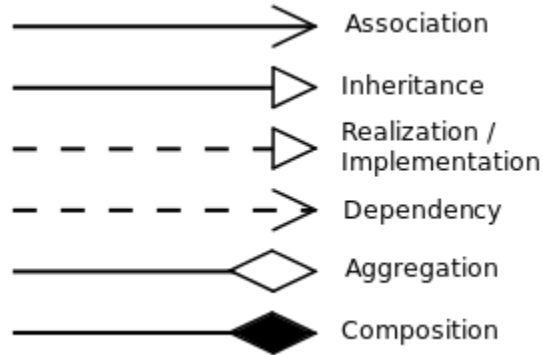
Perlu diingat bahwa UML awalnya dikembangkan untuk mendukung OOP, sehingga ada beberapa elemen yang mungkin tidak sesuai dengan paradigma prosedural. Misalnya, konsep pewarisan (*inheritance*) yang penting dalam OOP, tidak ada dalam paradigma prosedural.

A decorative border of green leaves and branches at the top of the slide.

2 – What is Komposisi (composition) *in* OOP ?

A decorative border of green leaves and branches at the bottom of the slide.

Types of Associations in OOP



- 1) Asosiasi biasa (*ordinary association*): objek memiliki hubungan yang terdefinisi secara jelas dan objek tersebut tidak bergantung satu sama lain.
- 2) Agregasi (*aggregation*): satu objek memiliki beberapa objek terkait, tetapi objek-objek tersebut dapat ada tanpa objek utama.
- 3) **Komposisi (*composition*)**: satu objek terdiri dari beberapa objek terkait, dan objek-objek tersebut hanya dapat ada dengan objek utama.
- 4) Inheritance (*pewarisan*): objek turunan mewarisi sifat dan perilaku dari objek induk.

Penggunaan asosiasi yang tepat dapat membantu mengorganisir kode program dan membuat program lebih mudah dipahami dan dikembangkan.

Composition *in* OOP

Composition

Noun

- ① the nature of something's ingredients or constituents; the way in which a whole or mixture is made up.
"the social composition of villages"

Synonyms:

makeup constitution configuration
structure construction conformation
formation form framework fabric
anatomy arrangement organization
format layout *informal:* setup

- ② a work of music, literature, or art.
"Chopin's most romantic compositions"

Synonyms:

work of art work creation
literary/musical/artistic work opus oeuvre
piece arrangement poem novel play
drama symphony concerto opera
painting drawing picture

Composite (gabungan)

Adjective

- ① made up of various parts or elements.
"this soup is one of those composite dishes which you gradually build up"

Synonyms:

compound complex combined blended
mixed compounded synthesized

Noun

- ① a thing made up of several parts or elements.
"the English legal system is a composite of legislation and judicial precedent"

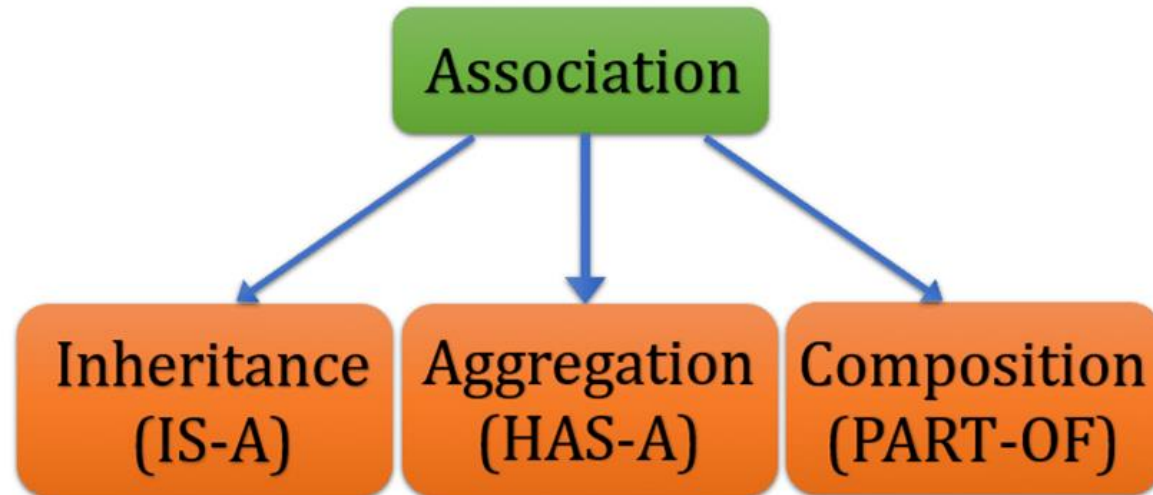
Synonyms:

amalgamation amalgam combination
compound fusion synthesis mixture
blend meld admixture conglomeration
alloy pastiche patchwork hybrid

Verb

- ① combine (two or more images) to make a single picture, especially electronically.

Composition in OOP

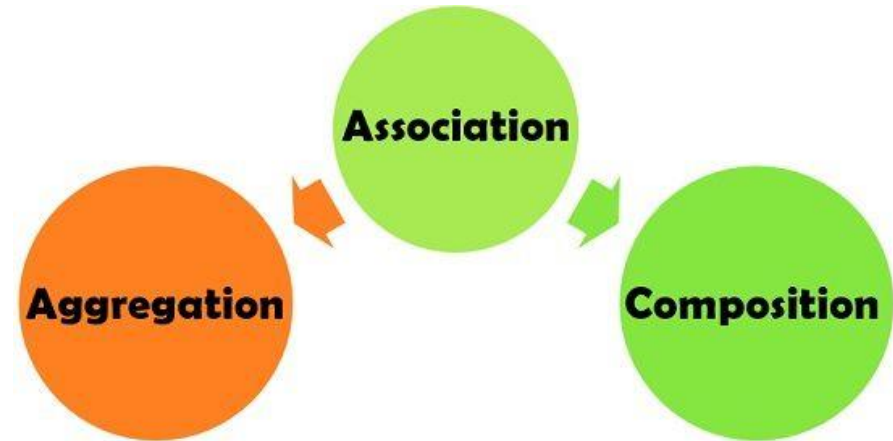
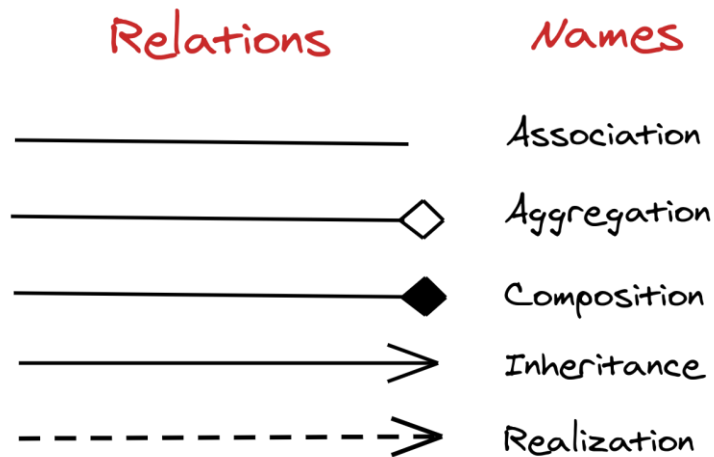


Association adalah hubungan antara dua objek yang menggambarkan bahwa satu objek terkait dengan objek lain. Objek-objek ini memiliki hubungan terpisah, dan keduanya dapat bertahan hidup tanpa satu sama lain.

Aggregation adalah jenis hubungan di mana satu objek berfungsi sebagai pemilik objek lainnya, dan objek-objek ini membentuk suatu kelompok. Objek yang dimiliki tidaklah menjadi milik objek pemilik, dan dapat dimiliki oleh objek lain.

Composition merupakan jenis hubungan yang serupa dengan Aggregation. Namun, perbedaannya adalah objek yang dimiliki tidak dapat berfungsi tanpa objek pemiliknya. Dalam komposisi, objek yang dimiliki menjadi milik objek pemilik, dan objek tersebut hanya dapat dimiliki oleh satu objek pemilik pada satu waktu.

Composition in OOP



Dalam Pemrograman Berorientasi Objek (OOP), **composition** adalah cara untuk membuat sebuah kelas dengan menggunakan referensi ke objek lain yang disebut **objek komponen**.

Objek komponen dalam composition adalah **objek yang digunakan oleh objek lain** untuk menyediakan fungsionalitas tertentu. Objek komponen biasanya dibuat terpisah dan kemudian digunakan oleh objek lain dalam struktur composition.

Struktur composition dalam Pemrograman Berorientasi Objek (OOP) mengacu pada **cara untuk menyusun objek-objek menjadi sebuah struktur hirarkis** dengan menggunakan objek komponen sebagai dasar penyusunannya. Dalam struktur composition, sebuah objek dibuat dengan menggunakan referensi ke objek-objek lain yang disebut objek komponen.

Composition *in* OOP

Dalam composition, objek komponen (*objek yang digunakan oleh objek lain*) dimiliki oleh objek yang memuatnya dan **tidak dapat eksis** di luar objek tersebut. Ini berarti bahwa ketika objek yang memuatnya dihancurkan, semua objek komponen yang ada di dalamnya juga akan dihancurkan.



Object Composition *vs* Object Aggregation

Composition in OOP (Object-Oriented Programming) is a technique for combining multiple objects into one more complex object and means that an object consists of one or more other objects. In composition, these objects cannot exist independently without a larger parent object.

Pada object composition,

objek yang satu diinisialisasi di dalam objek yang lain dan memiliki siklus hidup yang tergantung pada objek yang memuatnya.

Artinya, jika **objek induk dihapus**, maka objek anak akan dihapus juga.

Pada object aggregation,

objek yang satu bisa digunakan di objek yang lain tanpa menjadi bagian dari objek tersebut.

Artinya, objek yang satu tidak tergantung pada objek yang lain dan dapat hidup secara mandiri.

Composition OOP *with* Python

```
1 # class 1
2 class Mobil:
3     # parameter constructor
4     def __init__(self, jenis, warna):
5         self.jenis = jenis
6         self.warna = warna
7
8 # class 2
9 class Pemilik:
10     # parameter constructor
11     def __init__(self, nama, mobil):
12         self.nama = nama
13         self.mobil = mobil
14
15     # method 1
16     def tampilkan_data(self):
17         print("Nama Pemilik:", self.nama)
18         print("Jenis Mobil:", self.mobil.jenis)
19         print("Warna Mobil:", self.mobil.warna)
20
21 # Membuat objek Mobil
22 mobil1 = Mobil("Toyota Avanza", "Merah")
23
24 # Membuat objek Pemilik
25 pemilik1 = Pemilik("Semmy Taju", mobil1)
26
27 # Menampilkan informasi mobil beserta pemiliknya
28 pemilik1.tampilkan_data()
29
```

Pada composition, objek yang satu diinisialisasi di dalam objek yang lain dan memiliki siklus hidup yang tergantung pada objek yang memuatnya. Artinya, jika objek induk dihapus, maka objek anak akan dihapus juga.

Pada saat membuat objek *pemilik1*, kita menyertakan objek *mobil1* sebagai argumen. Ketika objek *pemilik1* dihapus, objek *mobil1* juga akan dihapus.

Aggregation OOP *with* Python

```
# class 1
class Mobil:
    # parameter constructor
    def __init__(self, merk, tahun):
        self.merk = merk
        self.tahun = tahun
        self.pemilik = None

    def display(self):
        print("Mobil:", self.merk, "Tahun:", self.tahun)
        if self.pemilik:
            self.pemilik.display()

# class 2
class Pemilik:
    # parameter constructor
    def __init__(self, nama, alamat):
        self.nama = nama
        self.alamat = alamat

    def display(self):
        print("Pemilik:", self.nama, "Alamat:", self.alamat)

# Membuat objek Pemilik
pemilik1 = Pemilik("Semmy Taju", "Jl. yordan No. 123")

# Membuat objek Mobil
mobil1 = Mobil("Toyota Avanza", 2023)

# Menghubungkan objek Mobil dengan objek Pemilik
mobil1.pemilik = pemilik1

# Menampilkan informasi mobil beserta pemiliknya
mobil1.display()
```

Pada aggregation, objek yang satu bisa digunakan di objek yang lain tanpa menjadi bagian dari objek tersebut. Artinya, objek yang satu tidak tergantung pada objek yang lain dan dapat hidup secara mandiri.

kita menghubungkan antara objek *mobil1* dengan objek *pemilik1* dengan melakukan assignment *mobil1.pemilik = pemilik1*.

Jika object *mobil1* di hapus maka object *pemilik1* tidak terganggu dan dapat hidup mandiri.



Exercise *for* Students

Vending Machine Minuman



Unklab Vending Machine *with* Python

```
1 # class 1
2 class FoodDrinkVendingMachine:
3     def __init__(self):
4         self.menu = []
5         self.payment = 0
6
7     def add_menu(self, item, price):
8         self.menu.append((item, price))
9
10    def display_menu(self):
11        print("Pilih menu yang kamu inginkan:")
12        for i, (item, price) in enumerate(self.menu):
13            print(f"{i+1}. {item} - Rp {price}")
14
15    def select_menu(self, choice):
16        item, price = self.menu[choice-1]
17        print(f"Kamu memilih {item} dengan harga Rp {price}")
18        return price
19
20    def insert_coin(self, coin):
21        self.payment += coin
22
23    def checkout(self, price):
24        if self.payment >= price:
25            change = self.payment - price
26            print(f"Pembayaran berhasil. Kembalian kamu adalah Rp {change}")
27            self.payment = 0
28        else:
29            print("Pembayaran tidak cukup. Silahkan masukkan uang lebih.")
30
31    def start(self):
32        print("Selamat datang di vending machine!")
33        self.display_menu()
34        choice = int(input("Masukkan nomor menu: "))
35        price = self.select_menu(choice)
36        coin = int(input("Masukkan uang: "))
37        self.insert_coin(coin)
38        self.checkout(price)
```

1

```
39
40
41    # switch case with loop and if/else
42    while True:
43        print("Pilih menu vending machine:")
44        print("1. Menu minuman dingin")
45        print("2. Menu minuman panas")
46        print("3. Menu makanan ringan")
47        print("0. Exit")
48        choice = int(input("Masukkan nomor menu vending machine: "))
49
50        if choice == 0:
51            break
52
53        if choice == 1:
54            machine = MenuColdDrink()
55        elif choice == 2:
56            machine = MenuHotDrink()
57        elif choice == 3:
58            machine = MenuSnackFood()
59        else:
60            print("Pilihan tidak valid.")
61            continue
62
63    # start vending machine
64    machine.start()
```

2

```
# class 2
class MenuColdDrink(FoodDrinkVendingMachine):
    def __init__(self):
        super().__init__()
        self.add_menu("Coca Cola", 5000)
        self.add_menu("Fanta", 5000)
        self.add_menu("Sprite", 5000)
        self.add_menu("Aqua", 3000)

# class 3
class MenuHotDrink(FoodDrinkVendingMachine):
    def __init__(self):
        super().__init__()
        self.add_menu("Kopi Panas", 5000)
        self.add_menu("Teh Panas", 3000)
        self.add_menu("Coklat Panas", 7000)

# class 4
class MenuSnackFood(FoodDrinkVendingMachine):
    def __init__(self):
        super().__init__()
        self.add_menu("Chitato", 5000)
        self.add_menu("Oreo", 6000)
        self.add_menu("Kit Kat", 7000)
        self.add_menu("Tango Wafer", 4000)
```

3

Unklab Vending Machine *with* Python

```
In [1]: runfile('D:/UNKLAB CLASSES/UNKLAB TEACHING/CLASS/Object-Oriented Programming/CODES/
Lecture25 [composition]/VendingMachine.py', wdir='D:/UNKLAB CLASSES/UNKLAB TEACHING/CLASS/Object-
Oriented Programming/CODES/Lecture25 [composition]')
```

Pilih menu vending machine:

1. Menu minuman dingin
2. Menu minuman panas
3. Menu makanan ringan
0. Exit

Masukkan nomor menu vending machine: 1

Selamat datang di vending machine!

Pilih menu yang kamu inginkan:

1. Coca Cola - Rp 5000
2. Fanta - Rp 5000
3. Sprite - Rp 5000
4. Aqua - Rp 3000

Masukkan nomor menu: 3

Kamu memilih Sprite dengan harga Rp 5000

Masukkan uang: 4000

Pembayaran tidak cukup. Silahkan masukkan uang lebih.

Pilih menu vending machine:

1. Menu minuman dingin
2. Menu minuman panas
3. Menu makanan ringan
0. Exit

Masukkan nomor menu vending machine:

Tidak ada **BATASAN** fungsi dalam pengembangan Vending Machine System (RMS) berbasis OOP sederhana ini, kalian bisa berkreasi dengan menambahkan fungsi ataupun fitur-fitur yang baru. Buatlah versi Vending Machine kalian sendiri.

Selain dalam bentuk REPORT, students juga akan diminta untuk menjelaskan alur proses setiap method dan demo di depan class !!!

Tambahkan daftar minuman dan makanan yang baru menjadi masing-masing 10 opsi.

END PRESENTATION

Thank you for your attention

Instructor: S – W – T

THANK YOU

